JVM Production Debugging — Study Sheet & Answering Framework

For Spring Boot on AWS ECS, targeting SDE-2 / Senior Engineer interviews

---

Part 1: The Answering Framework (use for every scenario question)

Say this structure out loud in the interview, even before you know the answer:

1. **Clarify the symptom** — what exactly changed (latency/CPU/errors/memory), and since when.
2. **Rank 2-3 hypotheses** — don't jump straight to a tool.
3. **Pick the tool that eliminates the most hypotheses fastest.**
4. **Correlate across signals** — one tool's output vs. a timeline (deploy, traffic spike, GC pause graph).
5. **State root cause + fix + prevention** (monitoring/alerting/load test/code guardrail).

Quick symptom → tool map:

Symptom	Most likely cause	First tool
Latency ↑, CPU flat	Blocking somewhere (GC, lock, I/O)	GC logs → JFR
CPU pegged, unclear why	Hot method / inefficient code	async-profiler CPU flame graph
Errors only under load	Thread pool / queue exhaustion	Thread dump + pool metrics
Full GC frequent, heap usage low	Explicit `System.gc()`, off-heap pressure, metaspace, GC tuning issue	GC logs
Slow memory creep over days	Memory leak	Heap dump (compare over time)
Weird/duplicate/skipped results under concurrency	Thread-safety bug (shared mutable state)	Code review + thread dump
One request affects others	Missing isolation / shared thread pool / shared resource	Thread dump + bulkhead review

---

Part 2: The Five Pillars

1. Java Flight Recorder (JFR)

What: Low-overhead (~1-2%) always-on profiler built into the JVM since Java 11 (backported to 8u). Records CPU samples, allocations, GC events, lock contention, thread starts, I/O — as timestamped events.

How to trigger:

```bash
# Start with the JVM
-XX:+FlightRecorder -XX:StartFlightRecording=duration=60s,filename=recording.jf

# Attach to a running process (no restart needed)
jcmd <pid> JFR.start duration=60s filename=recording.jfr
jcmd <pid> JFR.dump filename=snapshot.jfr
jcmd <pid> JFR.stop
```

Analyze: Open in JDK Mission Control (JMC) — free GUI. Look at:

- Hot Methods (CPU tab)
- Allocation by class (Memory tab)
- GC pauses (GC tab)
- Lock instances (Threads tab)

When to reach for it: Your default "give me everything" tool when you don't yet know if it's CPU, memory, or lock related. Good answer for the Java 11→21 migration question — JFR before/after comparison shows exactly what category shifted.

ECS gotcha: JFR file is written inside the container — you need to `aws ecs execute-command` in, or write to an EFS/S3-mounted volume, or use `jcmd ... JFR.dump` + `docker cp` / ECS Exec to pull it out.

---

2. Thread Dumps

What: A snapshot of every thread's stack trace + state at one instant.

How to capture:

```bash
jstack <pid> > threaddump.txt
# or, without needing jstack installed:
kill -3 <pid>          # goes to stdout/stderr
jcmd <pid> Thread.print
```

Take **3 dumps, 10 seconds apart** — a single dump can't distinguish "genuinely stuck" from "just slow." If the same threads are stuck at the same line across all three, that's your smoking gun.

Thread states to know cold:

- `RUNNABLE` — executing or ready to (could be CPU-bound OR doing blocking I/O — the JVM can't tell these apart, which is a common interview trap)
- `BLOCKED` — waiting to enter a `synchronized` block another thread holds
- `WAITING` / `TIMED_WAITING` — `Object.wait()`, `Thread.join()`, `LockSupport.park()`, pool waiting for work
- `NEW` / `TERMINATED`

Reading a dump for contention:

- One thread `BLOCKED` won't hurt you. **Hundreds of threads `BLOCKED` on the same monitor address** = a single lock is a bottleneck. Find the thread that is `RUNNABLE` and holds that lock ("locked <0x...> (a ...)") — that's your culprit, not the 400 threads waiting.
- Deadlock: jstack literally prints "Found one Java-level deadlock" with the cycle.

When to reach for it: `RejectedExecutionException` under load, hundreds of threads waiting on the same monitor, overlapping scheduled jobs, suspected pool exhaustion.

---

3. Heap Dumps

What: A full snapshot of every object on the heap + reference graph, at one instant.

How to capture:

```bash
jmap -dump:live,format=b,file=heap.hprof <pid>

# Better: auto-dump on OOM, always keep this on in prod
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/heap.hprof
```

Analyze with Eclipse MAT (Memory Analyzer Tool):

- Run "Leak Suspects" report first — it's automated and usually points at the right retained-size hog.
- **Dominator tree** — shows which single object is keeping the most memory alive. This is the #1 leak-hunting view.
- **Path to GC Roots** (exclude weak/soft refs) — shows *why* an object can't be collected, i.e., what's holding the reference chain. This answers "what's leaking" AND "who's leaking it."

Leak detection method: Take two heap dumps hours apart under similar load, compare object counts by class ("histogram" view). Classes with counts growing linearly with time/requests but never shrinking = leak candidates. Classic culprits: unbounded caches, static collections, listeners never deregistered, `ThreadLocal` never cleaned up in pooled threads.

When to reach for it: Slow memory creep, OOM crashes, suspected `ThreadLocal` leaks in long-running services (this directly answers your question #12).

---

4. GC Logs

What: Timestamped record of every GC event — what triggered it, how long it paused, how much was reclaimed.

Enable (Java 9+ unified logging):

```bash
-Xlog:gc*:file=/var/log/gc.log:time,uptime,level,tags:filecount=5,filesize=100M
```

Read for:

- **Pause time** — is it the pause itself, or the frequency, hurting you?
- **Minor/Young GC vs Full/Old GC** — frequent Young GC is often fine; frequent **Full GC** is the red flag.
- **Allocation rate** — how fast is young gen filling up? High allocation rate → GC runs constantly even if nothing is "leaking."
- **Promotion rate** — objects surviving into old gen too fast, either genuine long-lived objects or a young-gen sized too small.

Full GC every 8-10 min but heap usage never exceeds 55% (your sample Q3) — investigate:

- Explicit `System.gc()` calls in code or a library (RMI, NIO direct buffer cleanup pre-Java 9 does this) — check with `-XX:+DisableExplicitGC` as a diagnostic (not necessarily permanent fix).
- Metaspace/classloader churn (dynamic proxies, hot-reload frameworks) triggering Full GC as a side effect, not heap pressure itself.
- Off-heap/native memory pressure (direct ByteBuffers, native libs) triggering GC as the JVM's blunt "try to free something" response.
- CMS-specific: `CMS Initiating Occupancy Fraction` may be set too low, causing routine — not emergency — Full GCs.
- Scheduled/cron job doing something heavy exactly every 8-10 min — correlate the log timestamps against your job scheduler, not just the JVM.

ECS-specific GC gotcha: Container CPU limits ≠ visible host CPU. Before Java 10, the JVM didn't read cgroup limits and would size GC threads/heap based on the *host's* full CPU

count, causing GC to be starved of the CPU share it thinks it has. Java 10+ with `-XX:+UseContainerSupport` (default on) fixes this — but if you're troubleshooting an old image or a base image override, check this first.

---

5. async-profiler & Flame Graphs

What: Sampling profiler that (unlike JFR's default) can profile **native + Java frames together**, with low overhead, no safepoint bias (it uses `AsyncGetCallTrace`/perf_events, so it doesn't only sample at safepoints — this matters, because safepoint-biased profilers can hide the real hot method).

How to run:

```bash
./profiler.sh -d 30 -f flamegraph.html <pid>          # CPU profiling
./profiler.sh -e alloc -d 30 -f alloc.html <pid>       # allocation profiling
./profiler.sh -e lock -d 30 -f lock.html <pid>         # lock contention profil
```

Reading a flame graph:

- **X-axis = proportion of samples**, NOT time sequence. Width = how often that frame was on-CPU. Wide frame = hot.
- **Y-axis = call stack depth.** Bottom = entry point (e.g., `main` or thread run loop), top = leaf/innermost call.
- You're looking for **wide plateaus near the top** — a specific leaf method consuming a large % of total CPU samples, disproportionate to what it should be doing.
- **Differential flame graphs** (before/after a change) are the strongest tool for "did Streams actually slow this down" type questions — profile both versions, diff them, see exactly which frames changed width.

CPU vs lock vs context-switching (your sample Q8) — how to tell them apart:

- CPU-bound: `top -H` shows threads actually consuming CPU%; async-profiler CPU flame graph shows wide computational frames.
- Lock-bound: threads are `BLOCKED`/`WAITING` in thread dumps despite low CPU%; async-profiler `-e lock` shows time spent in monitor acquisition.
- Excessive context switching: high CPU in `top` but low *useful* work; check `vmstat` (`cs` column) or `pidstat -w`; often caused by too many threads relative to cores, or a pool sized far beyond what the workload needs.

Part 3: Threading/Concurrency Bugs (no tool needed — this is code review)

Several of your sample questions (#4, #7, #11) aren't really about profiling tools — they're testing whether you recognize classic concurrency bugs by pattern:

- **Skips records silently, no exception** → likely a shared non-thread-safe collection (e.g., `ArrayList`, `HashMap`) being mutated concurrently — corruption doesn't always throw, it can just silently lose entries or infinite-loop on resize. `HashMap` under concurrent `put()` is the textbook case (your Q11) — its resize can create a cycle in the bucket's linked list, causing an infinite loop or dropped entries, and it often only shows under real concurrent load, not in unit tests. Proving root cause: reproduce with a concurrent stress test (many threads hammering `put`), thread dump will show a thread spinning in `HashMap.resize` at high CPU (not blocked!), plus confirm the fix (`ConcurrentHashMap`) makes it disappear under the same test.
- **Duplicate responses to one request** → check for retry logic without idempotency (client retry + server actually processed the first one), or a race in a "check-then-act" cache/dedup mechanism that isn't atomic, or duplicate message delivery if there's a queue involved (at-least-once semantics).
- **One corrupted file slows down unrelated requests** → shared thread pool with no isolation (bulkhead pattern missing) — one slow/hanging task consumes a worker thread the whole pool needs. Fix: dedicated executor per resource type, timeouts, circuit breaker.
- **Scheduled job overlapping itself** → `@Scheduled(fixedRate=...)` without `fixedDelay` distinction, or missing distributed lock across ECS tasks (each task instance runs its own copy of the scheduler!). Fix: `ShedLock`/DB-based lock, or `fixedDelay`, or move to a single scheduler source (e.g., EventBridge triggering one task).

Part 4: ECS-Specific Talking Points (mention these — they show real production experience)

- **Multiple task instances = multiple independent JVMs.** A thread dump or heap dump from one task doesn't represent the whole fleet — you need to know *which* task instance is misbehaving (CloudWatch Container Insights per-task metrics) before you exec in.
- **Getting dumps out of a container:** `aws ecs execute-command` to shell in, then either `docker cp`-equivalent via S3 upload from inside the container, or mount an EFS volume for diagnostics.
- **CPU limits vs JVM ergonomics:** confirm `-XX:+UseContainerSupport` is active (default Java 10+); mismatched cgroup limits vs JVM's perceived CPU count causes wrong GC thread counts and wrong default heap sizing (`-XX:MaxRAMPercentage` matters more than `-Xmx` in containers).
- **CloudWatch Container Insights / X-Ray** as your first correlation layer before you even get a dump — latency/CPU/memory graphs over time tell you *when* to look, before you decide *what* to look with.

Part 5: Two Worked Examples

Q1: Latency +40% after Java 11→21 migration, CPU unchanged.

1. Clarify: is it p50, p99, or both? (Different causes.)
2. Hypotheses: (a) GC behavior changed — default collector assumptions, pause characteristics; (b) a library/agent incompatible with newer JIT/escape analysis, losing an optimization; (c) new blocking behavior — e.g., virtual threads pinning on `synchronized` blocks if you adopted them; (d) container CPU detection changed subtly between versions.
3. Tool: pull GC logs from before/after — compare pause frequency and duration first (fastest to rule in/out since CPU is flat, the extra time is likely *waiting*, not computing). Then JFR before/after, diff the "Hot Methods" and "GC" tabs.
4. Correlate: overlay latency graph against GC pause timestamps — if p99 latency spikes line up with GC pauses, that's your answer.
5. Fix: could be as simple as re-tuning `-Xmx`/GC flags for the new default collector, or finding a `synchronized` block causing virtual thread pinning if virtual threads were adopted.

Q6: Thread dump shows hundreds of threads `BLOCKED` on the same monitor.

1. Clarify: is this during a specific window (traffic spike) or constant?
2. Hypothesis: a single synchronized resource (cache, singleton service method, logger) is a bottleneck under concurrency.
3. Tool: in the thread dump, find the *one* thread that is `RUNNABLE` (not blocked) and holds that lock — jstack shows `locked <0xADDRESS>` on it. Look at its stack trace to find the actual synchronized method/block.
4. Correlate: check if that method does something slow inside the lock (I/O, DB call) — that's usually the real bug, not the locking itself.
5. Fix: narrow the critical section, replace with a concurrent data structure (`ConcurrentHashMap`, `ReadWriteLock`, or lock-free approach), or move the slow I/O outside the lock. Prevention: add a load test that specifically hammers this endpoint concurrently before merging changes to shared-state code.

Suggested Study Order

1. Thread dumps (cheapest to learn, highest question frequency)
2. GC logs (pairs with almost every "latency" question)
3. Heap dumps (leak questions)
4. JFR (ties everything together)
5. async-profiler (most tool-specific, but great differentiator vs other candidates)

Practice by taking a real thread/heap dump from a local Spring Boot app under JMeter load — reading a live one is 10x more useful than reading about it.